

Research Notes on Reinforcement learning human feedback

Reinforcement learning human feedback

>> Section 1

In machine learning, reinforcement learning from human feedback (RLHF) is a technique to align an intelligent agent with human preferences. It involves training a reward model to represent preferences, which can then be used to train other models through reinforcement learning. In classical reinforcement learning, an intelligent agent's goal is to learn a function that guides its behavior, called a policy. The function is iteratively optimized to increase the reward signal derived from the agent's task performance. However, explicitly defining a reward function that accurately approximates human preferences is challenging. Therefore, RLHF seeks to train a "reward model" directly from human feedback. The reward model is first trained in a supervised manner to predict if a response to a given prompt is good (high reward) or bad (low reward) based on ranking data collected from human annotators. This model then serves as a reward function to improve an agent's policy through an optimization algorithm like proximal policy optimization. RLHF has applications in various domains in machine learning, including natural language processing tasks such as text summarization and conversational agents, computer vision tasks like text-to-image models, and the development of video game bots. While RLHF is an effective method of training models to act better in accordance with human preferences, it also faces challenges due to the way the human preference data is collected. Though RLHF does not require massive amounts of data to improve performance, sourcing high-quality preference data is still an expensive process. Furthermore, if the data is not carefully collected from a representative sample, the resulting model may exhibit unwanted biases.

>> Section 2

The policy model is a function that takes a string as input, and produces another string. Usually in language modeling, the output string is not produced in one forward pass, but by multiple forward passes, generated autoregressively. Similarly to the reward model, the human feedback policy is also initialized from a pre-trained model. The key is to understand language generation as if it is a game to be learned by RL. In RL, a policy is a function that maps a game state to a game action. In RLHF, the "game" is the game of replying to prompts. A prompt and all previously generated tokens are the game state, and generating a new token is a game action. The first step in its training is supervised fine-tuning (SFT). This step does not require the reward model. Instead, the pre-trained model is trained on a dataset D_{SFT} that contains prompt-response pairs (x, y) . Then, during SFT, the model is

trained to auto-regressively generate the corresponding response y when given a random prompt x . The original paper recommends to SFT for only one epoch, since more than that causes overfitting. The dataset D_{SFT} is usually written by human contractors, who write both the prompts and responses. The second step uses a policy gradient method to the reward model. It uses a dataset D_{RL} , which contains prompts, but not responses. Like most policy gradient methods, this algorithm has an outer loop and two inner loops:

>> Section 3

where γ controls the strength of this pretraining term. This combined objective function is called PPO-ptx, where "ptx" means "Mixing Pretraining Gradients". It was first used in the InstructGPT paper. In total, this objective function defines the method for adjusting the RL policy, blending the aim of aligning with human feedback and maintaining the model's original language understanding. So, writing out fully explicitly, the PPO-ptx objective function is:

>> Section 4

Initialize the policy π_{ϕ}^{RL} to π^{SFT} , the policy output from SFT. Loop for many steps. Initialize a new empty dataset $D_{\pi_{\phi}^{\text{RL}}}$. Loop for many steps Sample a random prompt x from D_{RL} . Generate a response y from the policy π_{ϕ}^{RL} . Calculate the reward signal $r_{\theta}(x, y)$ from the reward model r_{θ} . Add the triple $(x, y, r_{\theta}(x, y))$ to $D_{\pi_{\phi}^{\text{RL}}}$. Update ϕ by a policy gradient method to increase the objective function
$$J(\phi) = E(x, y) - D_{\pi_{\phi}^{\text{RL}}}[r_{\theta}(x, y) - \beta \log \frac{\pi_{\phi}^{\text{RL}}(y|x)}{\pi^{\text{SFT}}(y|x)}]$$

>> Section 5

$$v(x, y) = \begin{cases} \lambda D_{\sigma}(\beta(r_{\theta}(x, y) - z_0)), & \text{if } y \sim y_{\text{desirable}} \\ \lambda U_{\sigma}(\beta(z_0 - r_{\theta}(x, y))), & \text{if } y \sim y_{\text{undesirable}} \end{cases}$$

Research Notes on Reinforcement learning human feedback

Reinforcement learning human feedback

$x \} \end{cases} \}$

>> Section 6

Next, invert this relationship to express the reward implicitly in terms of the optimal policy: $r^*(x, y) = \beta \log \frac{\pi^*(y|x) \pi^{\text{SFT}}(y|x) + \beta \log \pi^{\text{SFT}}(y|x)}{Z(x)}$.

>> Section 7

Direct preference optimization Direct preference optimization (DPO) is a technique to learn human preferences. Like RLHF, it has been applied to align pre-trained large language models using human-generated preference data. Unlike RLHF, however, which first trains a separate intermediate model to understand what good outcomes look like and then teaches the main model how to achieve those outcomes, DPO simplifies the process by directly adjusting the main model according to people's preferences. It uses a change of variables to define the "preference loss" directly as a function of the policy and uses this loss to fine-tune the model, helping it understand and prioritize human preferences without needing a separate step. Essentially, this approach directly shapes the model's decisions based on positive or negative human feedback. Recall, the pipeline of RLHF is as follows: